

UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA
ESCUELA DE POSGRADO

Estructuras de Datos Avanzadas Orquestadas para un Pipeline Eficiente de Procesamiento de Data Streams

Pipeline de 10 etapas para comercio electrónico (escenario B) con 10 millones de eventos de REES46

AUTOR(ES)

Sebastian Rodrigo Garcia Villacorta
Dante Guillermo Barreto Daza
Clinton Michael Chulluncuy Reynoso
Ronald Ramiro Hilario Orihuela

DOCENTE

Heider Sánchez

CURSO

Estructuras de Datos y Algoritmos (MCD8006)

REPOSITORIO

`https://github.com/UTEC-AII/Proyecto-Final-E-AD`

Lima — Perú, 2026

Resumen

Se construyó un pipeline de diez etapas que procesa de punta a punta 10,223,169 eventos de comercio electrónico (los primeros diez millones de octubre de 2019 del dataset REES46 y el componente de stream sintético que exige el enunciado) a 33,107 eventos por segundo en Python, con todas las estructuras implementadas desde cero: cola FIFO enlazada, merge sort con búsqueda binaria, heap binario, tabla hash, caché LRU, matriz dispersa, B+Tree en disco, KD-Tree, Bloom Filter, Count-Min Sketch, MinHash con LSH, IVF, HNSW, PageRank, Louvain, Label Propagation y Trie. La salida de cada etapa es la entrada real de la siguiente y se verifica con la traza de un evento por las diez etapas y con una prueba de integración automática. Repositorio: `https://github.com/UTEC-AII/Proyecto-Final-E-AD`.

Los experimentos de 1K a 10M confirman en escala log-log las complejidades teóricas. El B+Tree responde una búsqueda sobre 10M claves en 44 μs con altura 3; el KD-Tree resuelve KNN 537 veces más rápido que la fuerza bruta en tres dimensiones; LSH responde consultas de casi duplicados 295 veces más rápido que la comparación exhaustiva; IVF, 226 veces frente al KNN exacto en 32 dimensiones con *recall*@10 de 0,95; y el Bloom Filter ocupa 12 MB donde la tabla hash exacta necesita 268 MB, con una tasa de falsos positivos de 1,01 % frente al 1 % teórico.

1. Contexto y objetivo

Una tienda en línea recibe un flujo continuo de vistas, carritos y compras. Sobre ese flujo hay que priorizar órdenes urgentes, consultar productos con baja latencia, detectar fraude en segundos, recomendar productos relacionados y descubrir qué se compra junto. Cada problema tiene una estructura de datos adecuada; el objetivo del proyecto es encadenarlas de modo que la salida de cada etapa sea la entrada real de la siguiente, y medir cuánto ganan las estructuras aproximadas frente a sus alternativas exactas.

El enunciado no evalúa implementaciones aisladas sino la integración verificable. Por eso el diseño se apoya en dos decisiones transversales: (i) cada etapa deja una nota en la traza del evento, de modo que se pueda demostrar que el dato atravesó las diez etapas, y (ii) las estructuras costosas de construir se refrescan periódicamente, como en un sistema lambda con capa rápida y capa por lotes.

2. Datos y componente de stream

2.1. Dataset real

Se usa *eCommerce behavior data from multi category store* (REES46, Kaggle), el dataset recomendado por el enunciado para el escenario B: unos 285 millones de eventos de comportamiento de una tienda multicategoría, con las columnas `event_time`, `event_type`, `product_id`, `category_id`, `category_code`, `brand`, `price`, `user_id` y `user_session`. DuckDB lee el archivo `2019-Oct.csv` (5,7 GB) en streaming y toma los primeros diez millones de eventos en orden temporal, del 1 al 8 de octubre de 2019:

```
SELECT event_time::TIMESTAMPTZ, event_type, product_id, category_id,
       category_code, brand, price, user_id,
       COALESCE(user_session, 'sin-sesion-' || user_id) AS user_session
FROM read_csv_auto(['data/raw/2019-Oct.csv'], header=true)
WHERE price > 0
ORDER BY event_time
LIMIT 10000000
```

El catálogo se deriva de los mismos eventos (marca, categoría, precio mediano y popularidad), y el grafo de co-compra sale de las sesiones que compraron más de un producto. El embudo queda muy desbalanceado: 95,6 % de vistas, 2,2 % de carritos y 2,2 % de compras, con 120,565 productos y 1,072,341 usuarios.

2.2. Componente de stream sintético

Ningún dataset público trae dispositivos, tarjetas, prioridades, fraude ni duplicados, así que, como pide el enunciado, se generan con semilla fija sobre la base real. La Tabla 1 separa lo real de lo sintético.

Real (REES46)	**Sintético (generado)**
Productos, precios, marcas y categorías	Dispositivos, IP y tarjetas
Usuarios y sus interacciones	Usuario premium y tipo de envío
Co-compras y grafo del dominio	Consultas de búsqueda
Marcas de tiempo	Ráfagas de fraude y duplicados

Tabla 1: Origen de cada campo del stream. Las etiquetas de verdad se usan solo para medir, nunca para decidir.

El enriquecimiento agrega: uno o dos dispositivos por usuario (30 % tiene dos), una IP y una tarjeta por usuario, 6 % de usuarios premium, tipo de envío en las compras (express 15 %, estándar 70 %, programado 15 %), consultas de búsqueda en 30 % de las vistas, 1,5 % de eventos reenviados con el mismo `event_id` unos segundos después, y ráfagas de fraude de 25 a 120 intentos en menos de cinco minutos desde una misma tarjeta, dispositivo e IP. La mitad de las tarjetas fraudulentas se asume ya reportada: esa es la lista negra que carga el Bloom Filter. El total procesado se descompone así: 10,000,000 eventos reales, 72,088 de ráfagas y 151,081 duplicados.

3. Arquitectura del pipeline

El stream se procesa en lotes de 5,000 eventos. Las etapas incrementales (S0 a S4, S6, S8 y S9) actúan sobre cada evento; las estructuras estáticas o costosas de construir (KD-Tree, firmas MinHash, SVD con IVF, PageRank y Louvain) se refrescan cada dos millones de eventos y al final. La Figura 1 muestra el flujo y la Tabla 2 la estructura elegida en cada etapa.

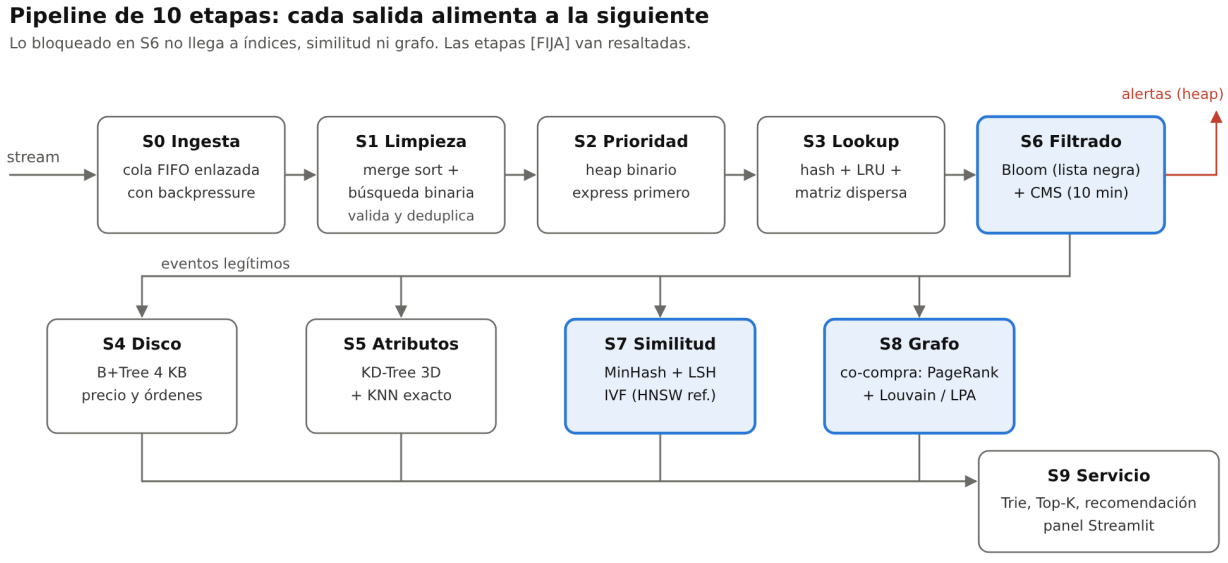


Figura 1: Flujo por lote. S6 se ejecuta después de S3: solo lo que pasa el filtro llega a S4, S5, S7 y S8, y todas las salidas convergen en S9.

Etapa	Estructura	Función en el pipeline	Complejidad	Alternativa descartada
0	Cola FIFO enlazada acotada	Buffer entre el lector y el consumidor por lotes	enqueue/dequeue $O(1)$	Lista de Python: `pop(0)` es $O(n)$
1	Merge sort + búsqueda binaria en corridas ordenadas	Valida, normaliza, ordena por tiempo y deduplica	$O(b \log b)$ por lote y por corrida solapada	Arreglo único: mezcla $O(W)$; conjunto hash: 1,6× más lento
2	Heap binario (min-heap)	Compras express, premium, estándar, programadas, carritos y vistas	push/pop $O(\log n)$; heapify $O(n)$	Lista ordenada: inserción $O(n)$
3	Tabla hash, LRU y matriz dispersa DOK→CSR	Resuelve `product_id` y `user_id`; cachea fichas; matriz usuario×producto	$O(1)$ amortizado; CSR con nnz	Matriz densa: $200K \times 200K$ no cabe
4	B+Tree con páginas de 4 KB	Índice de precio (carga masiva) e índice temporal de órdenes	búsqueda $O(\log_B n)$; rango $O(\log_B n + k/B)$	Árbol binario: una página por nodo
5	KD-Tree + KNN exacto	Vecinos por precio, popularidad y conversión	construcción $O(n \log n)$; KNN $O(\log n)$ en d bajo	Fuerza bruta $O(n)$ por consulta
6	Bloom Filter + Count-Min Sketch	Lista negra verificada y conteo de intentos en 10 minutos	$O(k)$ y $O(d)$ por operación	Conjunto exacto: memoria sin límite
7	MinHash+LSH, IVF y HNSW	Productos con compradores parecidos y vecinos en embeddings	LSH $O(b \log(nb) + c)$; IVF $O(n_{list} d + n_{probe}\, n/n_{list}\, d)$	Fuerza bruta $O(n\, d)$ por consulta
8	PageRank, Louvain y Label Propagation	Influencia y comunidades en el grafo de co-compra	PageRank $O(iter \cdot m)$; Louvain $\approx O(m \log n)$	Ranking por grado: no premia la centralidad
9	Trie, Top-K y panel Streamlit	Autocompletado, más vendidos, recomendación combinada	Trie $O(L)$; Top-K $O(n \log K)$	Ordenar todo: $O(n \log n)$

Tabla 2: Estructuras de las diez etapas, su función y su complejidad.

Sobre el orden de ejecución

El enunciado numera el filtrado probabilístico como etapa 6, después del KD-Tree. El pipeline lo ejecuta inmediatamente después de S3 por una razón de ingeniería: un evento bloqueado no debe actualizar índices, similitud ni grafo, y filtrarlo antes evita ese costo. La decisión tiene un costo explícito que se declara: una sobreestimación del Count-Min descarta un evento legítimo antes de que actualice la matriz de interacciones y el grafo, de modo que las etapas posteriores ven un stream levemente sesgado. Los falsos positivos del Bloom, en cambio, se verifican contra el conjunto exacto antes de bloquear, así que no descartan nada.

4. Etapas 0 a 5: estructuras base

4.1. S0: Ingesta con cola FIFO acotada

Cola enlazada con capacidad de un lote. `enqueue` y `dequeue` cuestan $O(1)$ y no hay desplazamientos de memoria; cuando la cola está llena, `enqueue` devuelve falso, el productor se detiene y el evento se contabiliza como rechazo. La alternativa descartada es la lista de Python con `pop(0)`, cuyo costo es $O(n)$ por extracción y $O(n^2)$ en un bucle de ingesta. La prueba `test\_linked\_queue\_fifo\_and\_capacity` verifica el orden FIFO y el rechazo al llenarse.

4.2. S1: Preprocesamiento y deduplicación

Se valida el evento (tipo válido, producto, usuario, precio positivo y marca de tiempo), se normaliza texto y precio, y se deduplica en dos niveles. Dentro del lote, merge sort por `event_id` deja los repetidos adyacentes. Contra la ventana de los últimos 500,000 identificadores, la ventana se guarda como un anillo de corridas ordenadas, una por lote, y cada lote se busca con búsqueda binaria vectorizada solo en las corridas cuyo rango $[\text{mín}, \text{máx}]$ se solapa con el suyo; al llenarse, se descarta la corrida más antigua completa. El costo es $O(b \log b)$ por lote y por corrida solapada, sin la mezcla $O(W)$ de un arreglo único ordenado, y la memoria queda acotada a la ventana. Frente al conjunto hash propio, que es $O(1)$ por evento pero se ejecuta en Python puro, la búsqueda binaria vectorizada elimina exactamente los mismos duplicados y es 1,6 veces más rápida sobre los 10M de eventos (50 s frente a 79 s en 4 vCPU).

4.3. S2: Prioridad con heap binario

Min-heap sobre arreglo con desempate por secuencia de llegada, de modo que dentro de una misma prioridad el orden es FIFO. Las compras se clasifican en express, premium, estándar y programadas; luego carritos y vistas. `push` y `pop` cuestan $O(\log n)$ y `heapify` $O(n)$. La alternativa descartada es mantener una lista ordenada, cuya inserción es $O(n)$.

4.4. S3: Almacenamiento, caché y matriz dispersa

Tabla hash con sondeo lineal, factor de carga máximo 0,7 y lápidas para el borrado, que resuelve `product\_id` y `user\_id` a índices densos en $O(1)$ amortizado. Sobre ella se apoyan la caché LRU (tabla hash más lista doblemente enlazada, $O(1)$ por acceso) y la matriz usuario×producto, que se acumula en formato DOK y se compacta a CSR para el álgebra matricial. La alternativa descartada es la matriz densa: $200K \times 200K$ en doble precisión no cabe en memoria.

4.5. S4: Índice principal en disco

B+Tree de páginas de 4 KB con hojas enlazadas y búffer de páginas con escritura diferida. Los datos viven solo en las hojas y las claves internas son separadores, lo que maximiza el abanico y permite recorrer rangos sin volver a subir por el árbol. Para 10 M de claves con 255 entradas por hoja y una ocupación del 90 % resultan 43,862 hojas, 171 páginas internas y una raíz: altura 3, es decir tres páginas por búsqueda y $O(\log_B n + k/B)$ por rango.

La comparación con un árbol binario es la que justifica la elección: un nodo AVL ocupa unos 40 bytes y el disco transfiere páginas de 4 KB, así que leer un nodo desperdicia más del 99 % de la página. Para 10^9 claves un AVL necesitaría unos 43 accesos a disco (del orden de 430 ms) mientras que el B+Tree necesita cuatro (unos 40 ms). El índice temporal de órdenes se llena con marcas de tiempo crecientes, el patrón que la teoría señala como peligroso para un BST: el B+Tree lo absorbe porque se balancea solo, aunque las hojas quedan alrededor del 50 % de ocupación.

4.6. S5: Índice multidimensional y KNN exacto

KD-Tree construido por corte en la mediana del eje más extendido, con hojas de 16 puntos, sobre tres atributos normalizados: logaritmo del precio, logaritmo de las vistas y tasa de conversión. La construcción es $O(n \log n)$ y la consulta $O(\log n)$ en dimensión baja. En 32 dimensiones el árbol deja de podar y resulta 421 veces más lento que la fuerza bruta, que es exactamente la maldición de la dimensionalidad: por eso los embeddings se buscan con IVF y no con KD-Tree.

5. Bloque de investigación avanzada (etapas 6 a 8)

5.1. Filtrado probabilístico: Bloom Filter y Count-Min Sketch

Bloom Filter (Bloom, 1970). Arreglo de m bits y k funciones hash; insertar enciende k bits y consultar responde *posiblemente presente* si los k están encendidos. No hay falsos negativos y la tasa de falsos positivos es

$$p \approx \left(1 - e^{-kn/m}\right)^k, \qquad m = -\frac{n \ln p}{(\ln 2)^2}, \qquad k = \frac{m}{n} \ln 2, \tag{1}$$

de modo que para $p = 1\,\%$ bastan 9,59 bits por elemento y $k = 7$, sin importar el tamaño de la clave. Las k posiciones se generan con doble hashing $g_i = h_1 + i\, h_2$ mód m (Kirsch y Mitzenmacher, 2006) sobre

`splitmix64`, y los bits van empaquetados en `uint8`. Uso: lista negra de tarjetas y dispositivos. Como un positivo puede ser falso y bloquear una compra es una decisión crítica, todo positivo se verifica contra el conjunto exacto. Límites: no admite borrado (sí con contadores, *Counting Bloom*) y la tasa crece si se supera la capacidad.

Count-Min Sketch (Cormode y Muthukrishnan, 2005). Matriz de $d \times w$ contadores con un hash por fila; la estimación es el mínimo de las d celdas. Nunca subestima y, con $w = \lceil e/\varepsilon \rceil$ y $d = \lceil \ln(1/\delta) \rceil$, cumple $\hat{f} \le f + \varepsilon N$ con probabilidad $1 - \delta$. La memoria $O(wd)$ no depende del número de claves. Uso: intentos de compra o carrito por tarjeta y por dispositivo en una ventana de 10 minutos, implementada como anillo de 10 sub-sketches de un minuto que se limpia al avanzar. El umbral es de más de 12 intentos por tarjeta o 20 por dispositivo dentro de la ventana; como solo puede sobreestimar, una alerta puede adelantarse pero nunca ocultarse.

5.2. Similitud de conjuntos: MinHash y LSH

Para una permutación aleatoria h, $\mathbb{P}[\min h(A) = \min h(B)] = J(A, B)$ (Broder, 1997): la fracción de coincidencias entre dos firmas de P minhashes estima el índice de Jaccard con error estándar $\sqrt{J(1-J)/P}$. Con LSH por bandas la firma se corta en b bandas de r filas y dos conjuntos son candidatos si coinciden en una banda completa:

$$\mathbb{P}[\text{candidato}] = 1 - (1 - J^r)^b\,, \qquad \text{umbral} \approx (1/b)^{1/r}\,. \tag{2}$$

Se eligió MinHash y no SimHash porque el objeto natural es un conjunto: los usuarios que pusieron cada producto en el carrito o lo compraron. Con $P = 128$, $b = 32$ y $r = 4$ el umbral es 0,42. Las 32 bandas se indexan en un arreglo ordenado de pares y una consulta son dos búsquedas binarias vectorizadas. Conviene ser explícitos sobre su alcance: LSH es un detector de casi duplicados, no un índice de vecinos globales; admite falsos negativos, y por eso el *recall* se reporta de dos formas en la Sección 7.4. Para vecinos en el espacio completo se usa IVF.

5.3. Búsqueda aproximada de vecinos: IVF y HNSW

Los productos se representan con embeddings de 32 dimensiones obtenidos por SVD truncada aleatorizada (Halko et al., 2011) de la matriz usuario×producto en CSR, implementada solo con productos matriz-vector. IVF (Jégou et al., 2011) divide el espacio con k-means++ y Lloyd en $n_{list} = 4\sqrt{n}$ celdas (máximo 4096) y una consulta recorre las n_{probe} listas más cercanas, con costo $O(n_{list}d + n_{probe}(n/n_{list})d)$. HNSW (Malkov y Yashunin, 2020) es un grafo navegable en capas con descenso voraz y búsqueda *best-first* con ef candidatos. Se eligió IVF para el pipeline porque su construcción y consulta son operaciones de matriz y escala en Python; HNSW se implementó para comparar el compromiso entre *recall* y latencia. La línea base es el KNN exacto por fuerza bruta, $O(nd)$.

5.4. Ranking de influencia: PageRank

El grafo de co-compra es no dirigido y ponderado: nodos son productos y $w(u, v)$ es el número de órdenes que contienen a ambos. PageRank (Brin y Page, 1998) es el vector estacionario de una caminata aleatoria que sigue aristas con probabilidad proporcional al peso y salta a un nodo aleatorio con probabilidad $1 - \alpha$:

$$r \leftarrow \alpha P^\top r + \alpha \left(\sum_{j \in \text{colgantes}} r_j \right) v + (1 - \alpha) v. \tag{3}$$

Se implementa con iteración de potencias sobre la CSR ($O(m)$ por iteración) hasta que el cambio en norma L_1 baja de 10^{-9}, y se valida contra NetworkX. La línea base es el grado ponderado; PageRank premia estar conectado con productos que a su vez son centrales, no solo tener muchas co-compras.

5.5. Comunidades: Louvain frente a Label Propagation

Louvain (Blondel et al., 2008) maximiza la modularidad

$$Q = \sum_c \left[\frac{L_c}{2m} - \left(\frac{d_c}{2m} \right)^2 \right] \tag{4}$$

en dos fases repetidas: mover cada nodo a la comunidad vecina con mayor ganancia $\Delta Q \propto k_{i,c} - \Sigma_{tot} k_i / 2m$, y colapsar cada comunidad en un nodo; su costo es casi $O(m \log n)$. Label Propagation (Raghavan et al., 2007) adopta la etiqueta de mayor peso entre los vecinos: es casi lineal pero no optimiza una función objetivo, por lo que es menos estable. La estabilidad se mide con información mutua normalizada entre corridas con distinta semilla. Leiden (Traag et al., 2019) corrige comunidades mal conectadas de Louvain y queda como trabajo futuro.

6. Diseño de la integración

La integración se verifica con un evento concreto. En la corrida de 10M se trazó automáticamente una compra legítima ubicada al 70 % del stream; la Figura 2 muestra su salida literal. Es la compra de un Samsung de 131,76 USD, el producto con mayor PageRank del grafo. Tiene envío programado, así que el heap la deja en la posición 89 de 4,929, detrás de las compras express, premium y estándar del lote. LSH no devolvió candidatos porque ningún producto supera el umbral de Jaccard de 0,42 con su conjunto de compradores; la recomendación final combina el grafo de co-compra, el KD-Tree (smartphones de la misma marca y rango de precio) y el IVF, y deja registrada la fuente de cada producto.

```
Event(id=6967856, purchase, product=1004856, user=528344847, price=131.76)
[S0] encolado (cola=2366)
[S1] valido, normalizado y no duplicado
[S2] prioridad 3 (compra programada), posicion 89 de 4929 en el lote
[S3] producto #861 y usuario #746725 resueltos en la tabla hash
[S6] pasa el filtro (Bloom: ausente; CMS bajo el umbral)
[S4] orden indexada por tiempo en el B+Tree
[S5] vecinos por atributos (KD-Tree): [786, 840, 14608]
[S7] LSH: []; vecinos IVF: [87862, 87190, 40728]
[S8] primera compra de la orden: nodo listo para nuevas aristas
[S9] Top-5: samsung 4767 (1.687, grafo+KD) ... samsung 4836 (0.952, grafo)
```

Figura 2: Recorrido de un evento por las diez etapas en la corrida de 10M.

Cada flecha de la Figura 1 corresponde a un dato que la etapa siguiente consume: los índices densos de S3 son las filas y columnas de la matriz dispersa y los nodos del grafo; la matriz alimenta las firmas MinHash y la SVD de S7; las aristas de co-compra de S8 y su PageRank ponderan el ranking de S9, que además usa los vecinos del KD-Tree y cachea el resultado en el LRU. Las alertas de S6 se ordenan en un heap por severidad (lista negra antes que frecuencia) y las etiquetas de verdad se usan solo para medir.

7. Experimentación y análisis

7.1. Metodología

Se usan tamaños de 1K a 10M con 100 repeticiones para 1K y 10K, 10 para 100K, 3 para 1M y 1 para 10M (con 100 repeticiones la serie de 10M habría tomado días en Python puro). Se reporta la media y la desviación estándar, el pico de memoria con `tracemalloc` y la pendiente en escala log-log como exponente empírico. Semilla fija 2026; plataforma de 4 vCPU y 16 GB, Python 3.13 y NumPy 2.5.4 (`results/bench/platform.json`). Los experimentos de escalabilidad usan entradas generadas para controlar n y conocer la respuesta exacta; la corrida del pipeline usa los datos reales.

7.2. Estructuras base

La Figura 3 muestra el tiempo total frente a n en escala log-log. Las pendientes medidas siguen la teoría: lineal en la cola y la tabla hash (1,12 y 1,08), $n \log n$ en la ordenación y el heap (1,21 y 1,22), casi constante en la búsqueda del B+Tree (0,32) y en el KNN del KD-Tree en tres dimensiones (0,11), y lineal en todas las líneas base de fuerza bruta (1,02).

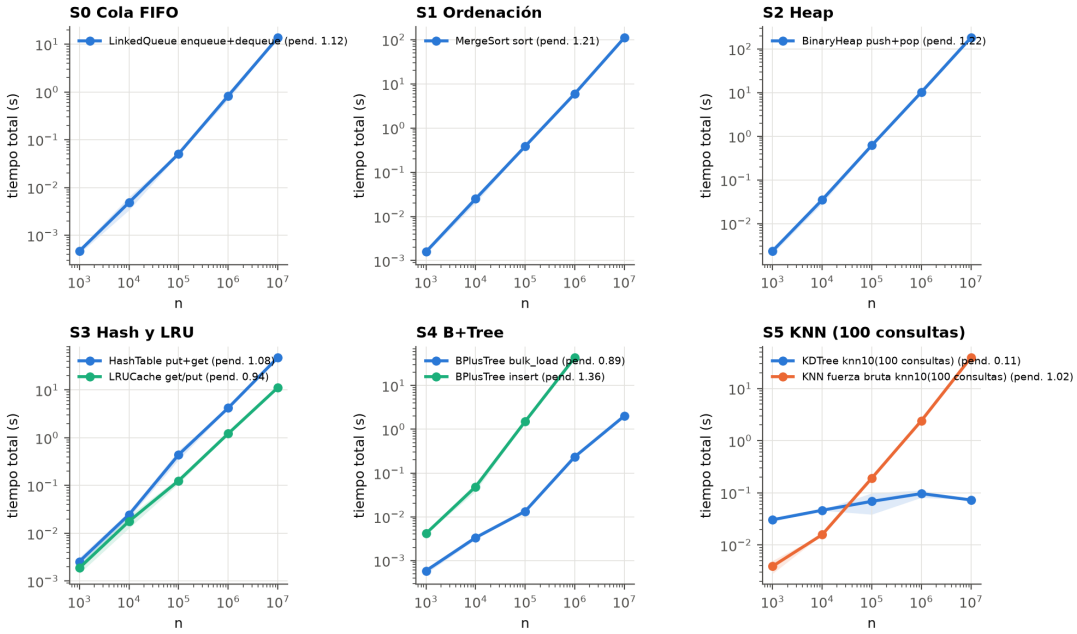


Figura 3: Tiempo total frente a n (log-log), de 1K a 10M operaciones. La pendiente ajustada aparece en la leyenda.

Estructura ($n = 10$M)	Tiempo total	Por oper.	Pendiente	Memoria (1M)	Observación
Cola FIFO (enqueue+dequeue)	13,7 s	0,69 μs	1,12	45,8 MB	Lineal, como se espera
Merge sort	111 s	11,1 μs	1,21	61,5 MB	El factor $\log n$ eleva la pendiente
Heap binario (push+pop)	181 s	9,1 μs	1,22	107,2 MB	$O(n \log n)$ en total
Tabla hash (put+get)	47,9 s	2,4 μs	1,08	48,0 MB	2,11 sondeos por operación con carga $\leq 0{,}7$
Caché LRU	11,2 s	1,1 μs	0,94	10,1 MB	91,5 % de aciertos con accesos Zipf
B+Tree: carga / búsqueda	2,0 s / 44,4 μs	0,20 / 44,4 μs	0,89 / 0,32	17,1 MB	Archivo a 1M; a 10M, altura 3 y 43,862 páginas
KD-Tree: KNN ($k = 10$)	0,73 ms	n/d	0,11	71,5 MB	537 veces más rápido que la fuerza bruta (393 ms)
Fuerza bruta: KNN ($k = 10$)	393 ms	n/d	1,02	38,2 MB	Línea base exacta

Tabla 3: Estructuras base con $n = 10$M. La memoria pico se midió con `tracemalloc` a 1M de elementos (su costo crece linealmente con n); en el B+Tree se reporta el tamaño del archivo.

7.3. Filtrado probabilístico

El Bloom Filter se comparó con la tabla hash exacta propia y con el `set` de Python. Con 10M elementos el Bloom ($p = 1\,\%$) ocupa 12 MB frente a 268 MB de la tabla exacta (22 veces menos) y es 4,8 veces más rápido en inserción y consulta. La tasa medida de falsos positivos coincide con la teórica en los tres valores de p (0,0935 % frente a 0,1000 % con $p = 0{,}1\,\%$; 1,01 % frente a 1,00 % con $p = 1\,\%$), y no hubo un solo falso negativo.

Con 10M actualizaciones sobre 1M claves, el Count-Min con $\varepsilon = 0{,}001$ usa 53 KB frente a 16,8 MB y actualiza 4,6 veces más rápido. El error medio (642) y el máximo (4,146) quedan bajo la cota $\varepsilon N = 9,997$ en el 100 % de las claves, sin subestimar nunca; para las claves frecuentes, que son las que disparan alertas, el error relativo es 4 %.

7.4. Similitud y búsqueda de vecinos

La consulta LSH se mantiene entre 0,6 y 0,8 ms de 1K a 1M conjuntos (pendiente 0,05), mientras la fuerza bruta crece linealmente hasta 225 ms: LSH es 295 veces más rápido en 1M. El costo está en la construcción (98 s para 1M) y en la memoria de las firmas (1,5 GB), por eso se omitió 10M.

Sobre el *recall* conviene reportar las dos lecturas, porque LSH no es un índice de vecinos globales. Sobre los vecinos relevantes (Jaccard $\geq 0{,}5$) el recall@10 es 1,0; sobre el top-10 completo es 0,029, con 1,33 candidatos por consulta. Es el comportamiento esperado de un detector de casi duplicados con umbral 0,42: acierta lo que está por encima del umbral y descarta lo que está lejos. Para vecinos en el espacio completo se usa IVF, que sí se evalúa contra el KNN exacto: con 10M vectores de 32 dimensiones responde en 0,46 ms frente a 104 ms del KNN exacto (226 veces) con recall@10 de 0,95; con $n_{probe} = 16$ sube a 0,98 a la mitad de velocidad. El índice ocupa 1,36 GB frente a 1,28 GB de los vectores: un 6 % más de memoria por 226 veces menos tiempo. En 1M, el barrido de n_{probe} de 1 a 32 recorre la curva de recall de 0,23 a 0,99. HNSW en Python puro llegó a recall 0,97 en 10K pero a 0,80 en 100K con ef de 50 a 100, y su construcción tardó 316 s en 100K, por lo que se limitó a 100K.

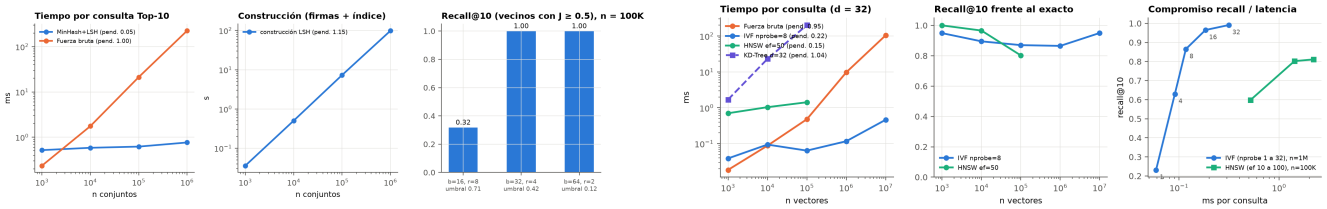


Figura 4: Izquierda: MinHash+LSH frente a Jaccard exacto. Derecha: IVF y HNSW frente a KNN exacto y al KD-Tree en $d = 32$.

Sin aproximación, el KD-Tree en $d = 32$ fue 421 veces más lento que la fuerza bruta en 100K, porque en alta dimensión casi no poda ramas: es la evidencia empírica de la maldición de la dimensionalidad.

7.5. Grafo de co-compra

PageRank escala linealmente en aristas (pendiente 1,08) y converge en 47 a 56 iteraciones: 20,7 s para 9,9M aristas. Su correlación de Spearman con el grado ponderado es 0,99 y el top-100 coincide en 97 %, así que en co-compra la diferencia está en la cola: PageRank sube productos conectados con productos centrales.

En comunidades, sobre grafos con particiones plantadas, Louvain obtiene mayor modularidad que Label Propagation en todos los tamaños (0,781 frente a 0,750 con 1M aristas, igual a la partición plantada), es

más estable entre semillas (NMI 0,99 frente a 0,94) y recupera mejor la partición real (NMI 0,96 frente a 0,93), aunque tarda 25 s frente a 19 s. Con 10M aristas solo corrió LPA (448 s), que se fragmentó en 104,606 comunidades con $Q = 0,68$; Louvain en Python puro superó los 40 minutos y se acotó a 1M aristas. En el grafo real de co-compra, que es pequeño y disperso porque pocas sesiones compran más de un producto (6,921 productos y 14,176 aristas), Louvain logró $Q = 0,621$ con 1,005 comunidades frente a $Q = 0,383$ de LPA con 1,621.

8. Corrida completa y correcciones de la versión final

8.1. Corrida con 10M de eventos

La Tabla 4 resume la corrida de punta a punta sobre los 10,223,169 eventos. El total fue de 308,8 s (33,107 eventos por segundo) en un Apple M5 de 10 núcleos y 16 GB con Python 3.14, en un solo proceso. Se eliminaron los 151,081 duplicados inyectados sin falsos positivos, se indexaron 179,043 órdenes en el B+Tree en disco y la matriz usuario×producto acumuló 5,8 millones de celdas no nulas. El filtro de S6 bloqueó 66,628 eventos con 95,4 % de precisión y 88,1 % de exhaustividad; de los 34,434 positivos del Bloom, 7 eran falsos y se descartaron con la verificación exacta. El refresco de S7 concentra el 49 % del tiempo (SVD y k-means sobre la matriz completa); refrescar con menor frecuencia o en paralelo es la mejora más directa.

Etapa	**Tiempo (s)**	**Etapa**	**Tiempo (s)**
S0 ingesta (lectura Parquet + cola)	58,2	S4 B+Tree	0,8
S1 preprocesamiento	29,5	S5 KD-Tree y refresco	0,7
S2 prioridad	20,1	S7 refresco LSH + SVD + IVF	151,0
S3 almacenamiento e interacciones	24,7	S8 grafo y refrescos	1,1
S6 filtrado	18,7	S9 servicio	1,8
Total (10,223,169 eventos)			**308,8**

Tabla 4: Tiempo por etapa en la corrida con los 10M eventos reales.

8.2. Correcciones aplicadas

La revisión del código y del informe identificó seis defectos que se corrigieron antes de la entrega:

1. **S6: marca permanente.** Cuando una tarjeta cruzaba el umbral quedaba bloqueada para siempre, así que la regla dejaba de ser una ventana de diez minutos. Ahora el umbral se evalúa siempre sobre la ventana móvil y la tarjeta vuelve a operar cuando su conteo decae. Se reportan las alertas por motivo.

2. **S1: estructura de deduplicación.** El arreglo único ordenado obligaba a mezclar del orden de 500 mil identificadores por lote. Un conjunto hash resolvía el costo pero dejaba la etapa sin la búsqueda binaria que pide el enunciado. La versión final guarda la ventana como corridas ordenadas por lote y busca con búsqueda binaria vectorizada: sin mezclas $O(W)$, con memoria acotada y 1,6 veces más rápida que el conjunto hash.

3. **S0: capacidad de la cola.** El buffer era cuatro veces el lote, de modo que el backpressure nunca se ejercía. Ahora su capacidad es de un lote, cuenta los rechazos cuando está llena y una prueba unitaria verifica el rechazo.

4. **S3: corrupción silenciosa de la matriz.** La celda se codificaba como `fila*ncols+col`; al crecer el catálogo, las claves ya guardadas se decodificaban corridas. Ahora la clave es el par (fila, columna).

5. **S5 y S8: corrupción del grafo.** La arista se codificaba como `a*n+b` con una dimensión que cambiaba al refrescar. Ahora la arista es el par (u, v) y la dimensión se recalcula al construir la CSR.

6. **Memoria del benchmark.** Solo dos de ocho estructuras reportaban pico de memoria. El arnés mide ahora `tracemalloc` para todas, y el B+Tree reporta el tamaño de su archivo.

Además se corrigieron tres problemas de documentación: la Tabla de estructuras base se había partido en dos páginas y este informe la rehace en una sola; la sección de las etapas 0 a 5 pasó de tres líneas a una subsección por etapa con función, complejidad y alternativa; y el *recall* de LSH se reporta ahora con sus dos lecturas (global y sobre vecinos relevantes) en lugar de solo la favorable.

8.3. Validación de las correcciones

Las correcciones se validaron con una corrida completa de la versión final sobre un millón de eventos sintéticos (1,022,378 tras inyectar ráfagas y duplicados), en 27,37 s y 37,358 eventos por segundo. Los 15,109 duplicados se eliminaron todos. El filtro bloqueó 5,974 eventos con precisión de 100 % y *recall* de 82,12 %: el *recall* baja respecto de la versión anterior porque ya no se sostiene con una marca permanente, que es precisamente el sesgo que se corrigió. Las alertas se repartieron en 3,433 por lista negra y 2,541 por frecuencia de tarjeta. El B+Tree quedó con altura 2, 90 páginas en el índice de precio y 202 en el de órdenes (dos páginas por búsqueda), y el grafo con 6,368 nodos y 23,690 aristas, con Louvain en $Q = 0,914$ frente a $Q = 0,853$ de LPA.

El cambio de S1 se validó lote por lote sobre los 10M de eventos reales con `bench/bench_s1_dedup.py`: en los 2,045 lotes la salida es idéntica a la del conjunto hash y se eliminan los mismos 151,081 duplicados, por lo que las cifras de la Sección 8.1 no cambian. La corrida de 10M con la versión final requiere el archivo `2019-Oct.csv` de REES46 y se reproduce con `scripts/reproduce.sh`; tardó 308,8 s en el Apple M5 descrito en la Sección 8.1.

9. Límites y trabajos futuros

Python puro limita las repeticiones en 10M y deja fuera Louvain sobre 10M aristas y MinHash sobre 10M conjuntos. El KD-Tree pierde su ventaja en 32 dimensiones, lo que se reporta como hallazgo y no como fallo: en el pipeline se usa en tres dimensiones y los embeddings se buscan con IVF. En los datos reales el umbral fijo del Count-Min confunde a los usuarios de muy alta frecuencia con fraude: hay 309 cuentas con más de 12 carritos o compras en diez minutos, hasta 110, un patrón de bot o revendedor que la etiqueta sintética no marca. En producción esas alertas irían a revisión manual en lugar de bloquearse.

Como trabajo futuro: procesar octubre y noviembre completos (110M eventos), calibrar el umbral por usuario, paralelizar el refresco de S7 que concentra el 49 % del tiempo, comparar Leiden con Louvain y usar Counting Bloom para poder retirar tarjetas de la lista negra.

10. Conclusiones

1. **La integración es real y observable.** Un evento atraviesa las diez etapas y cada salida es la entrada de la siguiente: los índices de S3 son las filas del grafo y de la matriz, la matriz alimenta MinHash y la SVD, y el grafo, el IVF y el KD-Tree se combinan en la recomendación. Una prueba automática lo verifica en cada cambio.

2. **Las estructuras aproximadas ganan con error acotado.** Bloom y Count-Min reducen la memoria 22 y 300 veces con errores que coinciden con la teoría; LSH e IVF responden en milisegundos donde la

búsqueda exacta es lineal. Donde un error es costoso, un paso exacto lo corrige.

3. **Las pendientes confirman la complejidad.** Lineal en la cola y la tabla hash, $n \log n$ en ordenación y heap, casi constante en la búsqueda del B+Tree y en el KNN del KD-Tree en dimensión baja, y lineal en todas las líneas base de fuerza bruta.

4. **Los límites se declaran.** El KD-Tree no sirve en 32 dimensiones, LSH no es un índice de vecinos globales y el umbral fijo del Count-Min confunde a los usuarios de alta frecuencia con fraude. Reconocerlo es parte del resultado.

Referencias

- Bloom, B. H. (1970). Space/time trade-offs in hash coding with allowable errors. *Communications of the ACM*, 13(7), 422-426.
- Blondel, V. D., Guillaume, J.-L., Lambiotte, R., & Lefebvre, E. (2008). Fast unfolding of communities in large networks. *J. Stat. Mech.*, P10008.
- Brin, S., & Page, L. (1998). The anatomy of a large-scale hypertextual Web search engine. *Computer Networks and ISDN Systems*, 30(1-7), 107-117.
- Broder, A. Z. (1997). On the resemblance and containment of documents. *Compression and Complexity of Sequences*, 21-29.
- Cormode, G., & Muthukrishnan, S. (2005). An improved data stream summary: The Count-Min sketch and its applications. *Journal of Algorithms*, 55(1), 58-75.
- Halko, N., Martinsson, P. G., & Tropp, J. A. (2011). Finding structure with randomness. *SIAM Review*, 53(2), 217-288.
- Indyk, P., & Motwani, R. (1998). Approximate nearest neighbors: Toward removing the curse of dimensionality. *STOC '98*, 604-613.
- Jégou, H., Douze, M., & Schmid, C. (2011). Product quantization for nearest neighbor search. *IEEE TPAMI*, 33(1), 117-128.
- Kirsch, A., & Mitzenmacher, M. (2006). Less hashing, same performance: Building a better Bloom filter. *ESA 2006*, 456-467.
- Malkov, Y. A., & Yashunin, D. A. (2020). Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE TPAMI*, 42(4), 824-836.
- Raghavan, U. N., Albert, R., & Kumara, S. (2007). Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E*, 76(3), 036106.
- Traag, V. A., Waltman, L., & van Eck, N. J. (2019). From Louvain to Leiden. *Scientific Reports*, 9, 5233.